



PYTHON PROGRAMMING

Python shutil Module

File and Folder Automation for Beginners

PRESENTED BY

Muhammd Yousif Memon

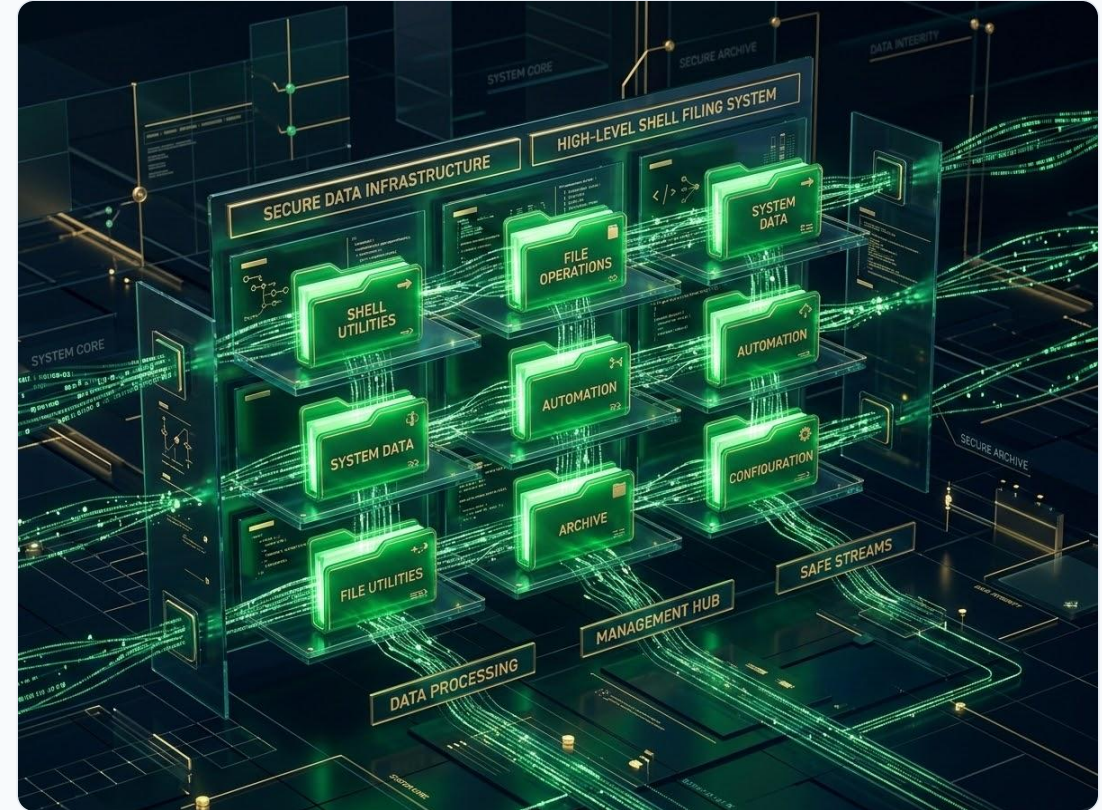
DATE OF SESSION

July 23, 2026

What is the shutil Module?

Understanding Shell Utilities in Python

- ✓ **Meaning & Standard Library:** The name `shutil` stands for "Shell Utilities," reflecting its purpose of providing shell-like file manipulation capabilities within Python. It comes pre-installed with Python, requiring no external packages via pip.
- ✓ **High-Level Operations & Scope:** Designed for copying, moving, and deleting entire collections of files, directories, and complex nested directory trees.
- ✓ **Contrast with Low-Level I/O:** Unlike low-level I/O operations (e.g., `open()`, `read()`, `write()`), which handle stream reading and writing of file contents, `shutil` deals with file structures, metadata, and the movement of data at the system level.



os, pathlib, and shutil

Comparing Python's File Management Modules



Architectural Relationship

In a production automation script, these modules often work together. **pathlib** is used to construct and validate paths, **os** might handle specific system-level checks, and **shutil** executes the heavy lifting of moving or copying the data.

Importing and Setting Up shutil

Best Practices for Path Management

```
# Import shutil and pathlib together
import shutil
from pathlib import Path
```

Relative vs. Absolute Paths

Absolute paths specify the full directory structure, making them safer and more reliable for background automation scripts where the working directory might change.

```
# Define absolute paths cleanly
src = Path('/path/to/source')
dst = Path('/path/to/destination')
```

Path Validation & Safety

Always verify paths and prepare target folders before running automation scripts.

1. Use **Path.exists()** to prevent errors:

```
if src.exists(): # Proceed safely
```

2. Pre-create parent folders automatically:

```
dst.parent.mkdir(parents=True, exist_ok=True)
```

Copying Files with shutil.copy()

Basic File Duplication

Purpose: Used to copy the contents and permission mode of a single file to a destination.

`shutil.copy(src, dst, *, follow_symlinks=True)`

src	The path to the file to be copied.
------------	------------------------------------

dst	The destination path. If dst is a directory, the file is copied into it with its original name. If it is a file path, it specifies the new name (enabling simultaneous renaming).
------------	--

Basic Automation & Renaming Example

```
import shutil
from pathlib import Path

src = Path('config.ini')
dst = Path('backup/config_backup.ini')

# Ensure backup directory exists
dst.parent.mkdir(parents=True, exist_ok=True)

shutil.copy(src, dst)
```

Preserving Metadata with shutil.copy2()

Retaining File History During Copies

- ✓ **Crucial Difference:** While `copy()` only copies data and permission mode, `copy2()` attempts to preserve all file metadata.
- ✓ **Understanding File Metadata:** Metadata includes creation time, modification time, system flags, and extended attributes.
- ✓ **Preserving Modification Timestamps:** `copy2()` ensures that the copied file retains the exact modification and access timestamps of the original source file.

When Preferred: Highly valued in incremental backup systems (where timestamps dictate changes) and forensic data preservation.



```
import shutil
# The copied file will have the same modification time as the original
shutil.copy2('important_data.csv', 'backup_drive/important_data.csv')
```


Copying File Permissions with `shutil.copymode()`

Managing Access Rights

File Permission Modes

- ✓ In Unix/POSIX systems, files have **read, write, and execute** permissions for the owner, group, and others.
- ✓ Use `shutil.copymode(src, dst)` to copy permission bits from the source to the destination file.

Content & Platform Rules

- ✓ Crucially, `copymode()` does **not** modify file content. It only updates the destination's permission bits.
- ✓ **Platform mapping:** Unix/POSIX fully supports permission structures, while Windows is limited to read-only attributes.

python example

```
import shutil

# Assuming 'script.sh' has execute permissions and 'new_script.sh' does not
shutil.copymode('script.sh', 'new_script.sh')

# 'new_script.sh' now has the same permissions as 'script.sh'
```

Copying File Metadata with `shutil.copystat()`

Comprehensive Status Transfer

Metadata Capabilities

`shutil.copystat(src, dst)` copies permission bits, last access time, last modification time, and flags from the source to the destination.

`copystat()` vs `copymode()`: While `copymode()` strictly transfers permission bits, `copystat()` synchronizes timestamps and system flags as well.

Data Preservation: Similar to `copymode()`, the actual data contents of the destination file remain completely untouched.

Practical Use Cases

Synchronization Utilities

Ideal for folder-mirroring and sync tools where you need a destination file to perfectly match the metadata of a source file after its contents have been updated independently.

Timeline Integrity

Ensures that administrative actions, backups, or automated build tasks preserve accurate historical access and modification timelines without rewriting actual content.

```
import shutil
shutil.copystat('source_file.txt', 'destination_file.txt')
```

600 × 400

Copying Complete Directories with shutil.copytree()

Recursive Folder Duplication

✓ Copying an Entire Directory Structure

`shutil.copytree(src, dst)` recursively copies an entire directory tree, including all nested folders and files.

✓ No Manual Loops Required

It handles the traversal of the directory structure automatically, saving you from writing complex recursive loops.

✓ Source and Destination Rules

By default, the destination directory **MUST NOT exist** beforehand. `copytree()` will create it.

✓ Using 'dirs_exist_ok' (Python 3.8+)

To merge folders safely into an existing directory, use `dirs_exist_ok=True`.

PRACTICAL BACKUP AUTOMATION

```
import shutil

# Copy entire project folder to a backup location
# dirs_exist_ok allows overwriting/merging

shutil.copytree(
    'my_project',
    'backup_v1',
    dirs_exist_ok=True
)
```



Destination Folder Constraint

Destination must not exist beforehand unless `dirs_exist_ok=True` is set (Python 3.8+).

Ignoring Files During Directory Copying

Selective Recursive Copies

✓ Using the ignore Parameter

`shutil.copytree()` accepts an ignore callable to filter which files and directories are copied.

✓ Using `shutil.ignore_patterns()`

The `shutil.ignore_patterns(*patterns)` factory function creates a callable that ignores files matching specific glob-style patterns.

✓ Practical Exclusions

- Temporary files: `*.tmp`
- System files: `.DS_Store`, `Thumbs.db`
- Cache folders: `__pycache__`

✓ Excluding Specific File Extensions

Useful for excluding large or unnecessary files, like `.log` or `.git` directories.

Creating Selective Backup Systems

```
import shutil

# Copy project, ignoring cache and git folders

shutil.copytree(
    'source_code',
    'clean_backup',
    ignore=shutil.ignore_patterns(
        '__pycache__',
        '.git',
        '*.tmp'
    )
)
```

Moving Files and Folders with shutil.move()

Relocating Data Efficiently

Signature: `shutil.move(src, dst)`

Moving Files and Folders

Efficiently moves files or complete directory trees to a new location.

Renaming Files and Folders

If the destination path specifies a new name, `move()` effectively renames the item during the move.

Behind the Scenes: How it Works

- ✓ **Same Filesystem:** Defaults to `os.rename()` for near-instantaneous operation.
- ✓ **Across Physical Drives:** Falls back to copying data (via `copy2()`) and then deleting the source.

<> `organize_downloads.py`

Organizing Downloaded Files Example

```
import shutil
from pathlib import Path

src = Path('downloads/report.pdf')
dst = Path('documents/reports/q3_report.pdf')

# Moves and renames the file
shutil.move(src, dst)
```

600 × 400

Deleting Directory Trees with `shutil.rmtree()`

Permanent Recursive Deletion



Safety Considerations

WARNING: This operation is permanent. It bypasses the system Trash/Recycle Bin. Data cannot be easily recovered.

Implementing Validations

```
import shutil
from pathlib import Path

target = Path('temp_build_files')
if target.exists() and target.is_dir():
    # Proceed with caution
    shutil.rmtree(target)
```

Removing Directories Recursively

`shutil.rmtree(path)` deletes an entire directory tree, including all contents.

Key Difference: `os.remove` vs `shutil.rmtree`

`os.remove()` Deletes a **single file**.

`shutil.rmtree()` Deletes a **folder and everything** inside it.

Common Use Cases

- ✓ Removing temporary folders generated during software builds or automation scripts.

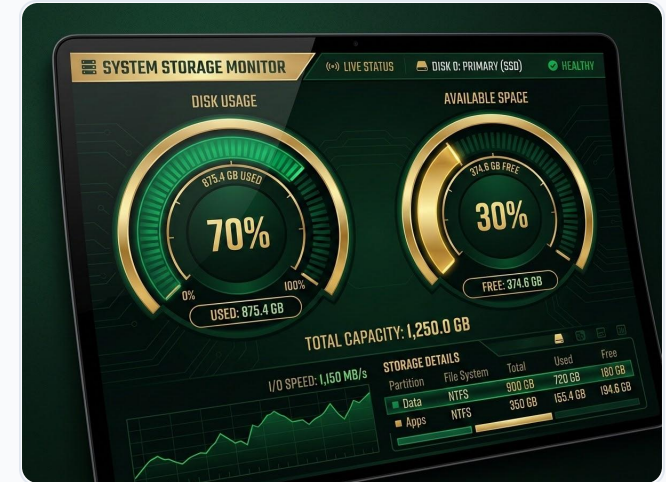
Working with Disk Usage

Using `shutil.disk_usage(path)` to monitor storage:

- ✓ **Returns a Named Tuple:** Provides (`total`, `used`, `free`) space measured in bytes.
- ✓ **Converting Bytes to Gigabytes (GB):** Essential for human-readable logs: `gb = bytes / (1024 ** 3)`
- ✓ **Practical Use Case:** Check free space before starting a heavy copy or backup operation to safely prevent disk-full errors.

```
import shutil
usage = shutil.disk_usage("/")
free_gb = usage.free / (1024 ** 3)
print(f"Free space: {free_gb:.2f} GB")

if free_gb < 10:
    print("Warning: Low disk space! Aborting backup.")
```



Finding Executable Programs with `shutil.which()`

Locating Command-Line Tools

✓ Purpose of `shutil.which(cmd)`

Searches the system PATH environment variable to find the executable file for a given command.

✓ Checking Dependencies

Useful for verifying if a required command-line tool (e.g., 'git', 'ffmpeg', 'python') is installed and accessible before running subprocesses.

✓ Handling Platform Differences

Automatically handles platform-specific executable extensions (e.g., appending `.exe` on Windows).

git_check.py

```
import shutil

git_path = shutil.which('git')

if git_path:
    print(f"Git found at: {git_path}")
else:
    print("Git is not installed or not in PATH.")
```

Creating Archives with `shutil.make_archive()`

Programmatic File Compression

What is File Archiving & Compression?

Bundling multiple files into a single archive file and reducing their overall storage footprint through compression.

Core Method Syntax

```
shutil.make_archive(base_name, format, root_dir)
```

Common Use Cases

- ✓ Compressing active project folders to save disk space dynamically.
- ✓ Creating automated, timestamped backup archives for deployment pipelines.



```
# Creates 'project_backup.zip' containing the contents of 'my_project' folder
import shutil
shutil.make_archive('project_backup', 'zip', 'my_project')
```


Automated Data Ingestion

✓ Extracting Archives

✔ Defining Custom Extraction Destinations

✔ Automated Ingestion Pipelines

✔ Error Safety

Python shutil Module: File and Folder Automation

Available Archive Formats

Supported Compression Types

Common Built-in Formats

- ✓ **zip** — Standard ZIP file
- ✓ **tar** — Uncompressed TAR archive
- ✓ **gztar** — GZIP compressed TAR archive
- ✓ **bztar** — BZIP2 compressed TAR archive
- ✓ **xztar** — XZ compressed TAR archive

Discovering Formats

Query your environment to view all archive formats currently supported on your target system:

```
shutil.get_archive_formats()
```

Customizing Formats

Register new archivers or customize existing formats dynamically by invoking the registration function:

```
shutil.register_archive_format()
```



Platform-Dependent Availability

Some compression formats may depend on the availability of underlying operating system utilities and libraries.

Handling Paths Safely

Cross-Platform Reliability

Best Practices: Using pathlib.Path

shutil functions accept pathlib.Path objects directly, which is highly recommended over raw strings.

Building Platform-Independent Paths

pathlib handles path separators automatically (/ on Unix, \ on Windows), preventing hard-coded path string errors.

Checking File Existence

Verify files exist and you have write permissions before starting operations to avoid crashes.

Avoiding Hard-Coded Path Strings

Use relative paths based on the script's location or configuration files rather than hard-coding absolute paths like C:\Users\...

Creating Destination Directories Dynamically

Create parent and sub-directories safely on-the-fly before writing files:

```
from pathlib import Path
dst_dir = Path('backups/2026')
# Creates 'backups' and '2026' if they don't exist
dst_dir.mkdir(parents=True, exist_ok=True)
```

Error Handling in File Automation

Building Robust Scripts

Why File Operations Fail: Operations can fail due to missing files, locked files, or disk space issues. Wrap critical file operations in robust blocks to prevent crashes.

Missing Files & OS Errors

FileNotFoundError: The source file or directory does not exist.

OSError: Broad category including disk full or disconnected network drives.

Permission & Logging Failures

PermissionError: The file is locked, read-only, or you lack admin rights.

Logging Failures: Log errors to a file instead of just printing to the console, allowing for post-mortem analysis.

```
import shutil
import logging

try:
    shutil.copy('source.txt', 'dest.txt')
except FileNotFoundError:
    logging.error("Source file not found.")
except PermissionError:
    logging.error("Permission denied.")
```

600 × 400

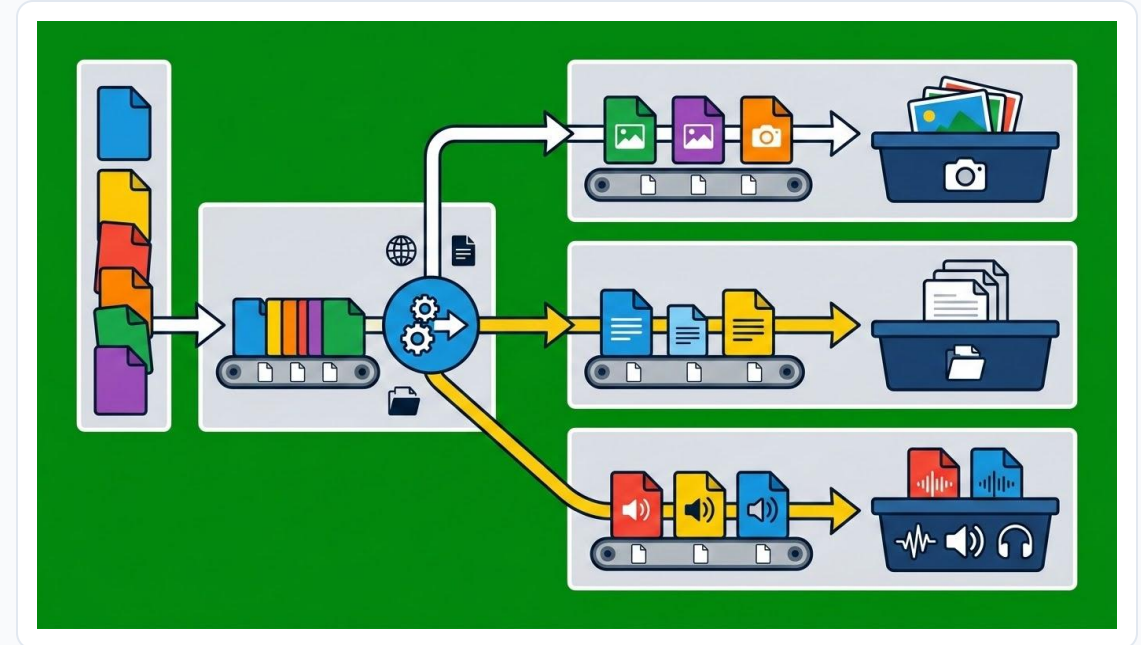
Automation Example: Automatic File Organizer

Scenario

Cleaning up a Downloads folder

- ✓ Scan files inside a directory using `pathlib`.
- ✓ Identify files by extension (e.g., `.pdf` Documents, `.png` Images).
- ✓ Create category folders dynamically if they don't exist using `mkdir(parents=True, exist_ok=True)`.
- ✓ Move files using `shutil.move()`.
- ✓ Handle unknown file types by moving them to an 'Others' folder.
- ✓ Prevent duplicate filename overwrites by renaming duplicates (e.g., adding a `_1` suffix before moving).

This script demonstrates combining path manipulation, directory creation, and file moving into a cohesive utility.



600 x 400

Automation Example: Automated Backup System

Practical Walk-Through

- 01 Initialize Configuration:** Define source directory and backup storage destination paths.
- 02 Name Generation:** Generate a timestamped archive name (e.g., `backup_2026_03_30.zip`).
- 03 Capacity Validation:** Check disk space using `shutil.disk_usage()` before starting to ensure sufficient capacity.
- 04 Archive Creation:** Compress the target directory into the specified format using `shutil.make_archive()`.
- 05 Storage Pruning:** Maintain only the 5 most recent archives using sorted lists and remove older ones with `os.remove` or `shutil.rmtree`.

Target Deployment Scenario

Designed to run as a reliable daily cron job or task-scheduler style script, operating autonomously in headless environments.



Resource-Aware Safety Strategy

This script actively manages system storage to prevent host volume exhaustion, checking disk capacity before archive initialization.

Guarantees continuous data safety while actively pruning outdated archive histories.

600 × 400

Automation Example: Temporary File Cleanup

Practical Walk-Through

Scenario: Safe cleanup utility for temp/cache files.

Safety Concept & Age Evaluation

- ✓ **Identify Targets:** Locate temporary directories and cache files (e.g., `__pycache__`, `build`, `.pytest_cache`).
- ✓ **Filter by Age:** Evaluate file modification time via `path.stat().st_mtime` to filter files older than 7 days.
- ✓ **Dry-Run Safeguard:** Performs a safe simulation by printing target directories first without executing actual removals.

```
if dry_run:
    print(f"Would delete: {path}")
else:
    shutil.rmtree(path)
```

Removal & Logging Execution

- ✓ **Exception-Safe Deletion:** Use `shutil.rmtree()` inside a robust `try-except` block to gracefully bypass locked files.
- ✓ **Storage Analytics:** Calculate directory sizes dynamically and log the total deleted storage space in Megabytes (MB).

```
try:
    shutil.rmtree(target_path)
    logger.info(f"Reclaimed {size_mb:.1f} MB")
except OSError as err:
    logger.warning(f"Locked: {err}")
```

i Key Takeaway: This script demonstrates robust, safe filesystem deletion practices combined with precise directory metadata evaluation.

Combining shutil with Other Python Modules

Building Comprehensive Tooling



shutil + os

Execute shell commands, manage environment variables, and change working directories around file operations.



shutil + pathlib

Achieve elegant, clean path handling and efficient directory scanning.



shutil + datetime

Append highly readable timestamps to files and archive names for version control.



shutil + logging

Generate clean, structured audit logs for automated tasks, essential for debugging unattended scripts.



shutil + schedule

Create an active background daemon to run cleanup or backup tasks hourly or daily without external cron jobs.

600 × 400

Mini Project: Smart File Management Automation

Bringing It All Together

STATUS	PHASE / FEATURE	FUNCTIONAL SPECIFICATION	CORE TECHNICAL LOGIC
✓	01 Monitor & Scan	Monitor/Scan: Target a specific folder on startup.	<pre>import os os.path.isdir(target_folder)</pre>
✓	02 Pre-Op Backup	Pre-operation Backup: Create a compressed backup of the target folder before making any changes using <code>make_archive</code> .	<pre>import shutil shutil.make_archive(zip_name, 'zip')</pre>
✓	03 Execution	Execution: Run a file categorization routine on the target folder.	<pre>for file in files: shutil.move(file, dest_dir)</pre>
✓	04 Reporting	Reporting: Output a formatted, detailed log file summarizing total files moved, space freed, and any skipped files due to <code>PermissionError</code> .	<pre>except PermissionError: log_skipped(file)</pre>
✓	05 Robustness	Absolute Robustness: A master <code>try-except</code> wrapper ensures the script terminates cleanly with a readable summary instead of raw Python stacktraces, even on critical failure.	<pre>try: main_routine() except Exception as e:</pre>

Summary and Best Practices

Key Takeaways for File Automation

Choosing the Right Function

`copy()` vs `copy2()` : Use `copy2()` to preserve metadata.

`copytree()` : Use for recursive folder duplication.

`move()` vs `pathlib.rename()` : Use `move()` for cross-drive operations.



Crucial Warning

`shutil.rmtree` is destructive, non-reversible, and bypasses the system trash bin. Use with extreme caution.



Validation is Key

Always validate storage availability with `disk_usage()` and verify paths with `exists()` before execution.



Enterprise-Grade Automation

Combine `shutil` operations with `logging` and comprehensive `try-except` blocks to create resilient scripts.



600 x 400